

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
"Высшая школа экономики"»

Московский институт электроники и математики им.
А.Н. Тихонова НИУ ВШЭ
Департамент компьютерной инженерии

Курс: Алгоритмизация и программирование

ОТЧЕТ
по лабораторной работе №5

Раздел	Максимальная оценка	Итоговая оценка
Работа программы	1	
Тесты	1	
Правильность алгоритма	3	
Листинг	0,5	
Ответы на вопросы	2	
Дополнительное задание	3	
Итого		

Студент: Солодилов Михаил Анатольевич
Группа: БИВ253
Вариант: 232 (5, 12, 2)
Руководитель:
Оценка:
Дата сдачи:

Оглавление

Задание	2
Внешняя спецификация	3
Листинг программы	4
Распечатка тестов к программе и результатов	8

Задание

1. Создать файл для хранения действительных чисел, вводимых с клавиатуры. Прочитать этот файл и вычислить максимальное среди отрицательных чисел.
2. Создать связанный список для хранения целых чисел. Число записей неизвестно, но можно ввести число, являющееся признаком окончания ввода данных. Для четных номеров вариантов организовать очередь, а для нечетных – стек. Для исходного списка решить следующую задачу: вставить заданное число $A1$ после каждого элемента с нечетным номером.
3. Для полученного списка решить следующую задачу: упорядочить по убыванию методом «пузырька» элементы списка, расположенные после первого положительного элемента.

Внешняя спецификация

Лабораторная работа №5

Задание №1

Введите действительные числа, ввод закончите пустой строкой:
< x >

До пустой строки
При $max = -\infty$

В последовательности не было отрицательных чисел

Иначе

$max = \ll max \gg$

Задание №2

Введите целые числа, ввод закончите пустой строкой:
< n >

До пустой строки

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Введите A1:
< $A1$ >

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

При $pos = NULL$

Нет положительных элементов

Иначе

Очередь:
 $\ll q[0] \gg, \ll q[1] \gg, \dots$

Листинг программы

```
1 #include <stdio.h>
2 #include <math.h>
3
4 static void read_stdin(const char* save_path);
5 static double find_max_neg(const char* path);
6
7 int main()
8 {
9     puts("Лабораторная работа №5");
10
11     puts("Задание №1");
12
13     read_stdin("data.txt");
14
15     double max = find_max_neg("data.txt");
16
17     if (max == -INFINITY)
18     {
19         puts("В последовательности не было отрицательных чисел");
20     }
21     else
22     {
23         printf("max = %lg\n", max);
24     }
25 }
26
27 static void read_stdin(const char* save_path)
28 {
29     FILE* save = fopen(save_path, "w");
30
31     char buffer[512] = "";
32
33     puts("Введите действительные числа, ввод закончите пустой строкой:");
34
35     fgets(buffer, sizeof(buffer), stdin);
36     while (buffer[1] != '\0')
37     {
38         double n = 0;
39         sscanf(buffer, "%lg", &n);
40         fprintf(save, "%lg\n", n);
41         fgets(buffer, sizeof(buffer), stdin);
42     }
43
44     fclose(save);
45 }
46
47 static double find_max_neg(const char* path)
48 {
49     FILE* f = fopen(path, "r");
50
51     char buffer[512] = "";
52
53     double max = -INFINITY;
54
55     while (fgets(buffer, sizeof(buffer), f))
56     {
57         double n = 0;
58         sscanf(buffer, "%lg", &n);
59
60         if (n < 0 && n > max)
61         {
62             max = n;
63         }
64     }
65
66     fclose(f);
67
68     return max;
69 }
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
```

```

4 typedef struct Node
5 {
6     int data;
7     struct Node* next;
8 } Node;
9
10 typedef struct Queue
11 {
12     Node* head;
13     Node* tail;
14 } Queue;
15
16 static void push(Queue* q, int n);
17 static void insert(Node* after, int n);
18 static void queue_dtor(Queue* queue);
19 static void print_queue(const Queue queue);
20 static Queue input_queue(void);
21 static void insert_after_odd(Queue* queue, int a1);
22
23 static Node* find_positive(Queue queue);
24 static void sort(Node* node);
25
26 int main()
27 {
28     puts("Лабораторная работа №5");
29
30     puts("Задание №2");
31
32     Queue q = input_queue();
33     puts("Очередь:");
34     print_queue(q);
35
36     int a1 = 0;
37     puts("Введите A1:");
38     scanf("%d", &a1);
39     insert_after_odd(&q, a1);
40     puts("Очередь:");
41     print_queue(q);
42
43     Node* pos = find_positive(q);
44     if (!pos)
45     {
46         puts("Нет положительных элементов");
47     }
48     else
49     {
50         sort(pos);
51         puts("Очередь:");
52         print_queue(q);
53     }
54
55     queue_dtor(&q);
56 }
57
58 static void push(Queue* q, int n)
59 {
60     if (q->tail == NULL)
61     {
62         q->tail = calloc(1, sizeof(Node));
63         q->tail->data = n;
64         q->head = q->tail;
65         return;
66     }
67
68     Node* new_node = calloc(1, sizeof(Node));
69     new_node->data = n;
70
71     q->tail->next = new_node;
72     q->tail = new_node;
73 }
74
75 static void insert(Node* after, int n)
76 {
77     Node* new_node = calloc(1, sizeof(Node));
78     new_node->data = n;
79     new_node->next = after->next;

```

```

80     after->next = new_node;
81 }
82
83 static void queue_dtor(Queue* queue)
84 {
85     Node* cur = queue->head;
86
87     while (cur)
88     {
89         Node* next = cur->next;
90         free(cur);
91         cur = next;
92     }
93
94     *queue = (Queue){};
95 }
96
97 static void print_queue(const Queue queue)
98 {
99     Node* cur = queue.head;
100
101     while (cur)
102     {
103         Node* next = cur->next;
104         printf("%d\n", cur->data);
105         cur = next;
106     }
107 }
108
109 static Queue input_queue(void)
110 {
111     Queue q = {};
112
113     char buffer[512] = "";
114
115     puts("Введите целые числа, ввод закончите пустой строкой:");
116
117     fgets(buffer, sizeof(buffer), stdin);
118     while (buffer[1] != '\0')
119     {
120         int n = 0;
121         sscanf(buffer, "%d", &n);
122         push(&q, n);
123         fgets(buffer, sizeof(buffer), stdin);
124     }
125
126     return q;
127 }
128
129 static void insert_after_odd(Queue* queue, int a1)
130 {
131     size_t n = 1;
132     Node* cur = queue->head;
133
134     while (cur)
135     {
136         Node* next = cur->next;
137         if (n % 2 == 1)
138         {
139             insert(cur, a1);
140         }
141         cur = next;
142         n++;
143     }
144 }
145
146 static Node* find_positive(Queue queue)
147 {
148     Node* cur = queue.head;
149
150     while (cur)
151     {
152         if (cur->data > 0)
153         {
154             return cur;
155         }

```

```

156         cur = cur->next;
157     }
158
159     return NULL;
160 }
161
162 static void sort(Node* node)
163 {
164     if (!node)
165     {
166         return;
167     }
168
169     Node* n1 = node;
170
171     while (n1)
172     {
173         Node* n2 = n1->next;
174         while (n2)
175         {
176             if (n1->data < n2->data)
177             {
178                 int t = n2->data;
179                 n2->data = n1->data;
180                 n1->data = t;
181             }
182             n2 = n2->next;
183         }
184         n1 = n1->next;
185     }
186 }

```


Распечатка тестов к программе и результатов

№	Исходные данные	Результат
1.1	$S[0 : 1] = \{0, 1\}$	В последовательности не было отрицательных чисел
1.2	$S[0 : 3] = \{5, 3, -4, 1\}$	$\max = -4$
1.3	$S[0 : 4] = \{5, 0, -6, -3, 1\}$	$\max = -3$
2.1	$S[0 : 2] = \{0, 1, 2\}, A_1 = 45$	Очередь: 0 45 1 2 45 Очередь: 0 45 45 2 1
2.2	$S[0 : 1] = \{-5, -3\}, A_1 = -100$	Очередь: -5 -100 -3 Нет положительных элементов